Heat Transfer Computer Project 1

Sichen Deng

March 6, 2026

1 Task A

1.1 Equation set with a clear unknown list

Unknown variables

- T_c : cavity gas temperature
- $T(r)$: radial temperature distribution in the solid layers
- $T_s = T(r_3)$: outer surface temperature
- Q : total heat transfer rate through the shell

1.1.1 Governing equations

For each solid layer $i = 1, 2, 3$, steady-state radial heat conduction in spherical coordinates is given by

$$\frac{1}{r^2}\frac{d}{dr}\left(r^2 k_i(T)\frac{dT}{dr}\right) = 0 \tag{1}$$

The three regions are:
Layer 1 (Al liner):

$$r_0 < r < r_1$$

Layer 2 (Aerogel):

$$r_1 < r < r_2$$

Layer 3 (CFRP):

$$r_2 < r < r_3$$

The temperature-dependent thermal conductivities are

$$k_1(T) = 160[1 - 2.0 \times 10^{-4}(T - 300)] \tag{2}$$
$$k_2(T) = 0.035 + 2.0 \times 10^{-4}(T - 300) + 1.0 \times 10^{-7}(T - 300)^2 \tag{3}$$
$$k_3(T) = 6.0[1 + 1.0 \times 10^{-4}(T - 300)] \tag{4}$$

1.1.2 Interface Conditions

Heat flux continuity:

$$-k_1(T)\frac{dT}{dr}\Big|_{r_1^-} = -k_2(T)\frac{dT}{dr}\Big|_{r_1^+} \tag{5}$$

$$-k_2(T)\frac{dT}{dr}\Big|_{r_2^-} = -k_3(T)\frac{dT}{dr}\Big|_{r_2^+} \tag{6}$$

Temperature jump due to contact resistance:

$$T(r_1^-) - T(r_1^+) = \frac{Q}{4\pi r_1^2} R''_{c,12} \tag{7}$$

$$T(r_2^-) - T(r_2^+) = \frac{Q}{4\pi r_2^2} R''_{c,23} \tag{8}$$

where

$$R''_{c,12} = 2.0 \times 10^{-4}\ m^2 K/W \tag{9}$$

$$R''_{c,23} = 1.0 \times 10^{-4}\ m^2 K/W \tag{10}$$

1.1.3 Inner Boundary Condition

The electronics generate a constant heat load

$$Q_{elec} = 50\ W \tag{11}$$

Energy balance in the cavity gives

$$Q_{elec} = h_i(T_c, T_1) A_0 (T_c - T_1) \tag{12}$$

where $T_1 = T(r_0)$, and

$$A_0 = 4\pi r_0^2 \tag{13}$$

The convection coefficient is obtained from

$$Ra = \frac{g_{eff}\beta |T_c - T_1| L_c^3}{\nu\alpha} \tag{14}$$

$$Nu = 2 + 0.55 Ra^{1/4} \tag{15}$$

$$h_i = \frac{Nu\, k_g}{L_c} \tag{16}$$

1.1.4 Outer Boundary Condition

The outer surface temperature is

$$T_s = T(r_3) \tag{17}$$

The radiation energy balance at the outer surface is

$$Q + A_3\alpha(T_s)(g_\odot G_\odot + g_M G_M) = A_3\varepsilon(T_s)\sigma(T_s^4 - T_{space}^4) \tag{18}$$

where $\sigma = 5.67 \times 10^{-8}\ W\, m^{-2} K^{-4}$, and

$$A_3 = 4\pi r_3^2 \tag{19}$$

$$\alpha(T_s) = 0.10 + 1.5 \times 10^{-4}(T_s - 300) \tag{20}$$

$$\varepsilon(T_s) = 0.92 - 1.0 \times 10^{-4}(T_s - 300) \tag{21}$$

1.2 Numerical solution approach

The steady-state model is solved numerically by discretizing the radial domain between r_0 and r_3 using a finite difference method. The spherical shell is divided into a set of nodes in the radial direction, and the governing heat conduction equation is applied at each node.

The temperature dependent thermal conductivities $k_i(T)$, the convection coefficient $h_i(T_c, T_1)$, and the radiation properties $\alpha(T_s)$ and $\varepsilon(T_s)$ introduce nonlinearities into the system of equations. Therefore, an iterative solution procedure is required.

In each iteration, the temperature-dependent properties are updated based on the current temperature estimates, and the resulting equations are solved repeatedly until convergence is achieved. Convergence is assumed when the maximum temperature change between successive iterations is smaller than 10^{-3} K.

1.3 Dominant Thermal Resistance

I think the dominant thermal resistance is the aerogel insulation layer (Layer 2). The thermal conductivity of aerogel is significantly lower than that of aluminum and CFRP, which from the conductivity values provided in the handout. Since thermal resistance increases with decreasing thermal conductivity, the conductive resistance of the aerogel layer is expected to dominate the overall heat transfer resistance. The contact resistances at the interfaces may cause temperature jumps but are expected to be smaller than the conduction resistance of the aerogel layer.

2 Task B

The numerical solution gives the following results for the baseline configuration:

- Cavity gas temperature: $T_g = 365.540$ K (FAIL)
- Inner wall temperature: $T_w = 331.149$ K
- Outer surface temperature: $T_s = 320.623$ K (PASS)
- Total heat transfer rate: $Q = 50.00$ W

The outer surface temperature satisfies the requirement $T_s < 340$ K, indicating that the spacecraft shell is able to reject heat effectively through radiation to space. The total heat transfer rate is also equal to the internally generated heat ($Q = 50$ W), confirming that the system reaches steady-state energy balance.

However, the cavity gas temperature is greater than the allowable limit of 365 K, so the design does not fully satisfy the thermal requirement.

The temperature drops across the contact resistances are:

$$\Delta T_{c12} = 0.00873 \text{ K} \tag{22}$$

$$\Delta T_{c23} = 0.00409 \text{ K} \tag{23}$$

which indicates that the thermal contact resistances between layers have a negligible impact on the overall heat transfer.

Real output from Python:

```
TASK B
Tg = 365.540 K, FAIL
Tw = 331.149 K
Ts = 320.623 K, PASS
Q = 50.00 W
dTc12 = 0.00873 K
dTc23 = 0.00409 K
Design: NOT SATISFIED
```

3 Task C

Figure 1 shows $T_g(t_2)$ with a line at 360 K. As the aerogel thickness t_2 increases, the thermal resistance of the insulation layer increases. As a result, the cavity gas temperature T_g increases because less heat can be conducted from the interior to the outer surface.

Figure 2 shows $T_s(t_2)$ with a line at 450K. The outer surface temperature T_s remains constant and stays well below the allowable limit of 450 K. This indicates that the external radiative heat rejection is not the limiting factor in the design.

From Fig. 1, the allowable thickness can be determined from the intersection between the T_g curve and the 360 K limit line. From the plot, this intersection occurs at approximately $t_2 = 4.33$ mm.

Real output from Python:

```
TASK C
t2_max = 4.33 mm
```

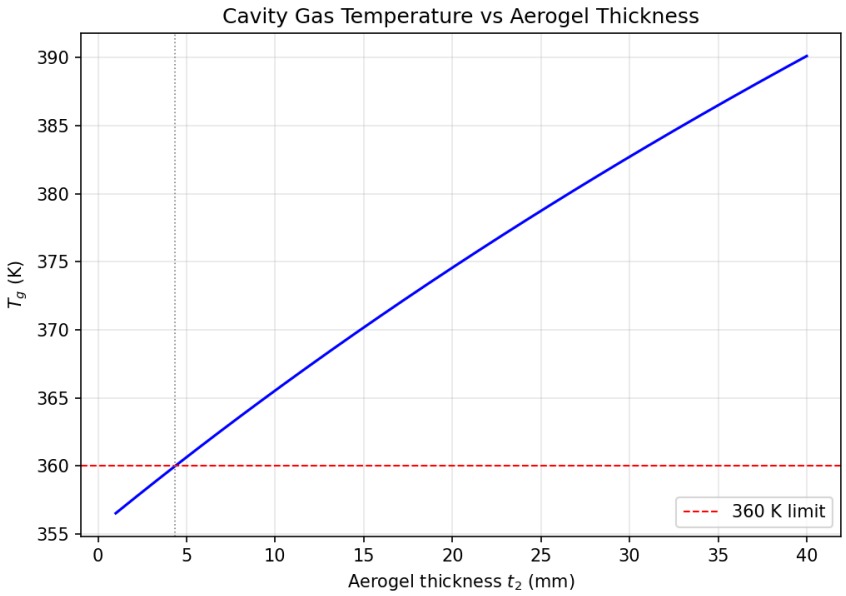

Figure 1: Cavity Gas Temperature depends on Aerogel Thickness

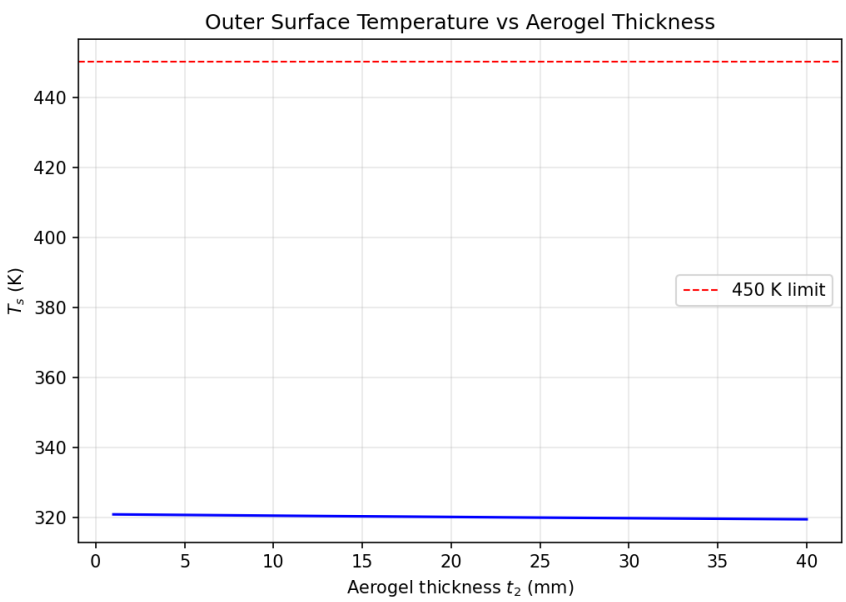

Figure 2: Outer Surface Temperature depends on Aerogel Thickness

4 Task D

The temperature distribution (Figure 3) shows that most of the temperature drop occurs across the aerogel insulation layer, which is shown as a red line on the plot. This is expected because aerogel has a very low thermal conductivity compared with the aluminum liner and the CFRP outer layer.

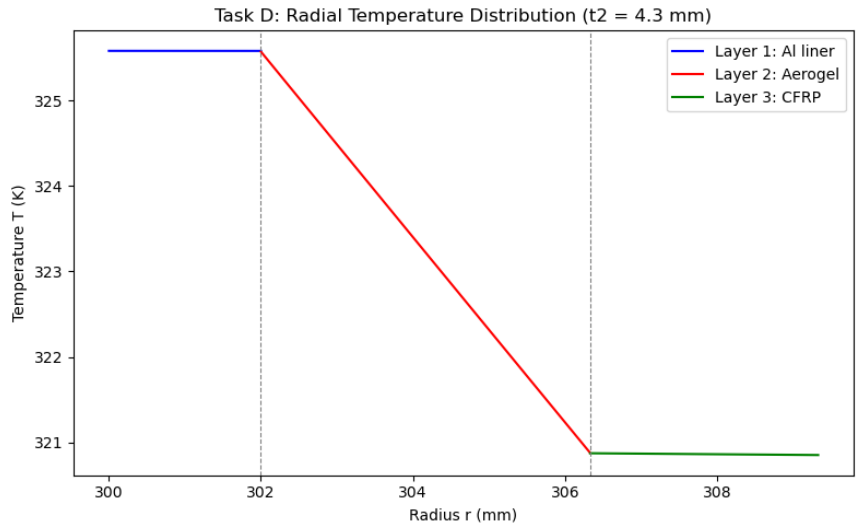

Figure 3: Radial Temperature Distribution across the three layers of the spacecraft shell at $t_2 = 4.33mm$

And the two interface jump magnitudes are:

$$\Delta T_{c12} = 0.00873 \text{ K} \tag{24}$$
$$\Delta T_{c23} = 0.00409 \text{ K} \tag{25}$$

Real output from Python:

```
TASK D
dTc12 = 0.00873 K
dTc23 = 0.00424 K
```

5 Task E

Figure 4 shows the variation of the cavity gas temperature T_g as a function of the electronics power Q_{int} for the design thickness $t_2 = 4.33\,\text{mm}$.

From the plot, the intersection between the T_g curve and the 360 K limit occurs at $Q_{int} = 50.04\,\text{W}$, which is the $Q_{int,max}$.

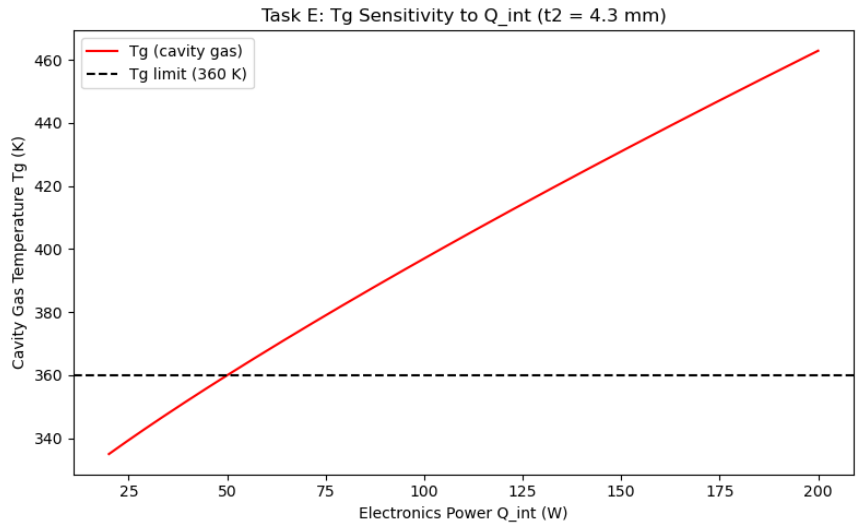

Figure 4: Cavity Gas Temperature T_g depends on Internal Electronics Power Q_{int} at $t_2 = 4.33mm$

Real output from Python:

```
TASK E
Q_int_max (Tg=360K) = 50.04 W
```

A Python Code

A.1 Base Model

```python
import numpy as np
from scipy.optimize import brentq

# Geometry [m]
R0 = 0.30        # inner cavity radius
T1 = 0.002       # Layer 1
T3 = 0.003       # Layer 3
T2_MIN = 0.001   # aerogel thickness range
T2_MAX = 0.040

def get_radii(t2):
    r1 = R0 + T1
    r2 = r1 + t2
    r3 = r2 + T3
    return R0, r1, r2, r3

```

```python
# Temperature dependent conductivities
def k1(T):
    return 160.0 * (1.0 - 2.0e-4 *(T - 300.0))

def k2(T):
    dT = T - 300.0
    return 0.035 + 2.0e-4 *dT + 1.0e-7 * dT**2

def k3(T):
    return 6.0 * (1.0 + 1.0e-4 * (T - 300.0))

# Contact resistances [m^2*K/W]
RC12_PP = 2.0e-4  # Al / Aerogel at r1
RC23_PP = 1.0e-4  # Aerogel / CFRP at r2

Q_INT_DEFAULT = 50.0  # electronics waste heat [W]

# Inner boundary: given gas propertise
KG = 0.030
NU_GAS= 1.7e-5
ALPHA_GAS = 2.4e-5
BETA = 1.0 / 300.0
LC = R0
G_EFF = 0.02

def h_inner(Tg, Tw):
    Ra = G_EFF * BETA * abs(Tg - Tw) * LC**3 / (NU_GAS * ALPHA_GAS)
    Nu = 2.0 + 0.55 * Ra**0.25
    return Nu * KG / LC

# Outer boundary: radiation
SIGMA = 5.670e-8
T_SPACE = 3.0
G_SOL = 9100.0
G_SOL_FAC  = 0.25
G_MERC= 1200.0
G_MERC_FAC = 0.25

def alpha_s(T):
    return 0.10 + 1.5e-4 * (T - 300.0)

def eps_s(T):
    return 0.92 - 1.0e-4 * (T - 300.0)

def Q_rad_net(Ts, r3):
    A = 4.0 * np.pi * r3**2
    a, e = alpha_s(Ts), eps_s(Ts)
```

```python
    return A * (e * SIGMA * (Ts**4 - T_SPACE**4)
                - a * G_SOL_FAC * G_SOL
                - e * G_MERC_FAC * G_MERC)

def solve(t2, Q_int=Q_INT_DEFAULT, N=20, tol=1e-6, max_iter=500, omega=1.0):
    r0, r1, r2, r3 = get_radii(t2)
    Q = Q_int

    # Outer surface temperature from radiation balance
    Ts = brentq(lambda T: Q_rad_net(T, r3) - Q, 100.0, 1500.0)

    # FDM node arrays for each layer
    r_L1 = np.linspace(r0, r1, N)
    r_L2 = np.linspace(r1, r2, N)
    r_L3 = np.linspace(r2, r3, N)
    dr1 = r_L1[1] - r_L1[0]
    dr2 = r_L2[1] - r_L2[0]
    dr3 = r_L3[1] - r_L3[0]

    # Unknowns: [Tg, T1[0..N-1], T2[0..N-1], T3[0..N-1]]
    n_total = 1 + 3 * N
    i1 = 1
    i2 = N + 1
    i3 = 2 * N + 1

    T_vec = np.zeros(n_total)
    T_vec[0] = Ts + 40
    T_vec[i1:i1+N] = np.linspace(Ts + 30, Ts + 20, N)
    T_vec[i2:i2+N] = np.linspace(Ts + 20, Ts + 5,  N)
    T_vec[i3:i3+N] = np.linspace(Ts + 5,  Ts,      N)

    converged = False
    for iteration in range(max_iter):
        T_old = T_vec.copy()

        Tg_c = T_old[0]
        T1c = T_old[i1:i1+N]
        T2c = T_old[i2:i2+N]
        T3c = T_old[i3:i3+N]

        A = np.zeros((n_total, n_total))
        b = np.zeros(n_total)

        # cavity gas energy balance
        hi = h_inner(Tg_c, T1c[0])
        a0 = hi * r0**2
```

```
        A[0, 0]  =  a0
        A[0, i1] = -a0
        b[0] = Q_int / (4.0 * np.pi)

        # layer 1
        r_ph = 0.5 * (r_L1[0] + r_L1[1])
        k_ph = k1(0.5 * (T1c[0] + T1c[1]))
        cp = r_ph**2 * k_ph / dr1
        A[i1, 0] = a0
        A[i1, i1] = -(a0 + cp)
        A[i1, i1+1] = cp

        # Interior nodes
        for i in range(1, N - 1):
            r_ph = 0.5 * (r_L1[i] + r_L1[i+1])
            r_mh = 0.5 * (r_L1[i] + r_L1[i-1])
            kp = k1(0.5 * (T1c[i] + T1c[i+1]))
            km = k1(0.5 * (T1c[i] + T1c[i-1]))
            cp = r_ph**2 * kp / dr1
            cm = r_mh**2 * km / dr1
            row = i1 + i
            A[row, row - 1] = cm
            A[row, row] = -(cm + cp)
            A[row, row + 1] = cp

        # Node N-1 at r1
        r_mh = 0.5 * (r_L1[N-2] + r_L1[N-1])
        km = k1(0.5 * (T1c[N-2] + T1c[N-1]))
        cm = r_mh**2 * km / dr1
        cr = r1**2 / RC12_PP
        row = i1 + N - 1
        A[row, row - 1] =cm
        A[row, row] = -(cm + cr)
        A[row, i2] =cr

        # Layer 2
        r_ph = 0.5 * (r_L2[0] + r_L2[1])
        kp = k2(0.5 * (T2c[0] + T2c[1]))
        cp = r_ph**2 * kp / dr2
        cr = r1**2/RC12_PP
        A[i2, i1 + N - 1] =  cr
        A[i2, i2]  = -(cr + cp)
        A[i2, i2 + 1] = cp

        # Interior nodes
        for i in range(1, N - 1):
            r_ph = 0.5 * (r_L2[i] + r_L2[i+1])
```

```
            r_mh = 0.5 * (r_L2[i] + r_L2[i-1])
            kp = k2(0.5* (T2c[i] + T2c[i+1]))
            km = k2(0.5 * (T2c[i] + T2c[i-1]))
            cp = r_ph**2 * kp / dr2
            cm = r_mh**2 * km / dr2
            row = i2 + i
            A[row, row -1] =  cm
            A[row, row] = -(cm + cp)
            A[row, row + 1] =  cp

        # Node N-1 at r2
        r_mh = 0.5 * (r_L2[N-2] + r_L2[N-1])
        km = k2(0.5* (T2c[N-2] + T2c[N-1]))
        cm = r_mh**2 * km / dr2
        cr = r2**2 / RC23_PP
        row = i2 + N - 1
        A[row, row -1] =  cm
        A[row, row] = -(cm + cr)
        A[row, i3] = cr

        # Layer 3 (CFRP)
        r_ph = 0.5 * (r_L3[0] + r_L3[1])
        kp = k3(0.5 * (T3c[0] + T3c[1]))
        cp = r_ph**2 * kp / dr3
        cr = r2**2/RC23_PP
        A[i3, i2 + N-1] = cr
        A[i3, i3] = -(cr + cp)
        A[i3, i3 + 1] = cp

        # Interior nodes
        for i in range(1, N - 1):
            r_ph = 0.5 * (r_L3[i] + r_L3[i+1])
            r_mh = 0.5 * (r_L3[i] + r_L3[i-1])
            kp = k3(0.5 * (T3c[i] + T3c[i+1]))
            km = k3(0.5 * (T3c[i] + T3c[i-1]))
            cp = r_ph**2 * kp / dr3
            cm = r_mh**2 * km / dr3
            row = i3 + i
            A[row, row -1] = cm
            A[row, row] = -(cm+ cp)
            A[row, row + 1] = cp

        # Node N-1 at r3; Ts
        row =i3 + N - 1
        A[row, row] = 1.0
        b[row] = Ts

```

```python
        T_new =np.linalg.solve(A, b)
        T_vec = omega * T_new + (1.0 - omega) * T_old

        if np.max(np.abs(T_vec -T_old)) < tol:
            converged = True
            break

    if not converged:
        print(f"warning: Picard did not converge in {max_iter} iterations ")

    Tg = T_vec[0]
    T1_arr = T_vec[i1:i1+N]
    T2_arr = T_vec[i2:i2+N]
    T3_arr = T_vec[i3:i3+N]

    Tw  = T1_arr[0]
    T1m = T1_arr[-1]
    T1p = T2_arr[0]
    T2m = T2_arr[-1]
    T2p = T3_arr[0]

    dTc12 = T1m - T1p
    dTc23 = T2m- T2p

    r_profile = np.concatenate([r_L1, r_L2, r_L3])
    T_profile = np.concatenate([T1_arr, T2_arr, T3_arr])

    return {
        'Tg': Tg, 'Tw': Tw,
        'T1m': T1m, 'T1p': T1p,
        'T2m': T2m, 'T2p': T2p,
        'Ts': T3_arr[-1], 'Q': Q,
        'dTc12': dTc12, 'dTc23': dTc23,
        'r0': r0, 'r1': r1, 'r2': r2, 'r3': r3,
        't2': t2, 'Q_int': Q_int,
        'r_profile': r_profile, 'T_profile': T_profile, 'N': N,
        'iterations': iteration + 1,
    }

def temperature_profile(sol):
    return sol['r_profile'], sol['T_profile']

if __name__ == '__main__':
    sol = solve(t2=0.010)
```

A.2 Plot Tasks

```python
import sys
import numpy as np
import matplotlib.pyplot as plt
import cp1_base as base


def task_B():
    print("\nTASK B")
    sol = base.solve(t2=0.010)

    print(f"Tg = {sol['Tg']:.3f} K, {'PASS' if sol['Tg'] <= 360 else 'FAIL'}")
    print(f"Tw = {sol['Tw']:.3f} K")
    print(f"Ts = {sol['Ts']:.3f} K, {'PASS' if sol['Ts'] <= 450 else 'FAIL'}")
    print(f"Q = {sol['Q']:.2f} W")
    print(f"dTc12 = {sol['dTc12']:.5f} K")
    print(f"dTc23 = {sol['dTc23']:.5f} K")

    both_pass = (sol['Tg'] <= 360) and (sol['Ts'] <= 450)
    print(f"Design: {'BOTH PASS' if both_pass else 'NOT SATISFIED'}")
    return sol


def task_C():
    print("\nTASK C")
    t2_arr = np.linspace(base.T2_MIN, base.T2_MAX, 200)
    Tg_arr = np.zeros_like(t2_arr)
    Ts_arr = np.zeros_like(t2_arr)

    for i, t2 in enumerate(t2_arr):
        sol = base.solve(t2)
        Tg_arr[i] = sol['Tg']
        Ts_arr[i] = sol['Ts']

    feasible = (Tg_arr <= 360.0) & (Ts_arr <= 450.0)
    t2_max = t2_arr[feasible][-1] if feasible.any() else None

    if t2_max is not None:
        print(f"t2_max = {t2_max*1e3:.2f} mm")
    else:
        print("No t2 found")

    fig1, ax1 = plt.subplots(figsize=(7, 5))
    ax1.plot(t2_arr * 1e3, Tg_arr, 'b-', linewidth=1.5)
```

```python
    ax1.axhline(360, color='r', linestyle='--', linewidth=1, label='360 K limit')
    if t2_max is not None:
        ax1.axvline(t2_max * 1e3, color='gray', linestyle=':', linewidth=0.8)
    ax1.set_xlabel('Aerogel thickness $t_2$ (mm)')
    ax1.set_ylabel('$T_g$ (K)')
    ax1.legend()
    ax1.grid(True, alpha=0.3)
    fig1.tight_layout()
    fig1.savefig('task_C_Tg.png', dpi=150)

    fig2, ax2 = plt.subplots(figsize=(7, 5))
    ax2.plot(t2_arr * 1e3, Ts_arr, 'b-', linewidth=1.5)
    ax2.axhline(450, color='r', linestyle='--', linewidth=1, label='450 K limit')
    ax2.set_xlabel('Aerogel thickness $t_2$ (mm)')
    ax2.set_ylabel('$T_s$ (K)')
    ax2.legend()
    ax2.grid(True, alpha=0.3)
    fig2.tight_layout()
    fig2.savefig('task_C_Ts.png', dpi=150)

    plt.show()
    return t2_max


def task_D(t2_max):
    print(f"\nTASK D")
    sol = base.solve(t2_max, N=100)
    r_arr, T_arr = base.temperature_profile(sol)
    N = sol['N']

    print(f"dTc12 = {sol['dTc12']:.5f} K")
    print(f"dTc23 = {sol['dTc23']:.5f} K")

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(r_arr[:N] * 1e3, T_arr[:N], 'b-', label='Layer 1: Al liner')
    ax.plot(r_arr[N:2*N] * 1e3, T_arr[N:2*N], 'r-', label='Layer 2: Aerogel')
    ax.plot(r_arr[2*N:] * 1e3, T_arr[2*N:], 'g-', label='Layer 3: CFRP')
    ax.axvline(sol['r1'] * 1e3, color='gray', linestyle='--', linewidth=0.8)
    ax.axvline(sol['r2'] * 1e3, color='gray', linestyle='--', linewidth=0.8)
```

```python
    ax.set_xlabel('Radius r (mm)')
    ax.set_ylabel('Temperature T (K)')
    ax.legend()
    plt.tight_layout()
    plt.savefig('task_D.png')
    plt.show()
    return sol


def task_E(t2_max):
    print(f"\nTASK E")
    Q_arr = np.linspace(20.0, 200.0, 200)
    Tg_arr = np.zeros_like(Q_arr)

    for i, Q_int in enumerate(Q_arr):
        sol = base.solve(t2_max, Q_int=Q_int)
        Tg_arr[i] = sol['Tg']

    idx = np.searchsorted(Tg_arr, 360.0)
    if 0 < idx < len(Q_arr):
        frac = (360.0 - Tg_arr[idx-1]) / (Tg_arr[idx] - Tg_arr[idx-1])
        Q_int_max = Q_arr[idx-1] + frac * (Q_arr[idx] - Q_arr[idx-1])
        print(f"Q_int_max (Tg=360K) = {Q_int_max:.2f} W")
    else:
        Q_int_max = None
        print("Tg does not reach 360K in range")

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(Q_arr, Tg_arr, 'r-', label='Tg (cavity gas)')
    ax.axhline(360, color='k', linestyle='--', label='Tg limit (360 K)')
    ax.set_xlabel('Electronics Power Q_int (W)')
    ax.set_ylabel('Cavity Gas Temperature Tg (K)')
    ax.legend()
    plt.tight_layout()
    plt.savefig('task_E.png')
    plt.show()
    return Q_int_max


def _get_t2_max():
    t2_arr = np.linspace(base.T2_MIN, base.T2_MAX, 200)
    t2_max = t2_arr[0]
    for t2 in t2_arr:
        sol = base.solve(t2)
```

```
        if sol['Tg'] > 360.0:
            break
        t2_max = t2
    return t2_max


if __name__ == '__main__':
    run = sys.argv[1].upper() if len(sys.argv) > 1 else 'BCDE'

    if 'B' in run:
        task_B()

    t2_max = None
    if 'C' in run:
        t2_max = task_C()

    if 'D' in run:
        if t2_max is None:
            t2_max = _get_t2_max()
        task_D(t2_max)

    if 'E' in run:
        if t2_max is None:
            t2_max = _get_t2_max()
        task_E(t2_max)
```